

Manual de uso — módulo calculator (Python)

Calculadora que interpreta expresiones en forma de tupla y devuelve el resultado. Puerto en Python del proyecto Java equivalente. Soporta operaciones escalares (binarias y unarias), ecuaciones polinómicas, listas de primos, aritmética de polinomios y aritmética de matrices.

1. Instalación y ejecución

Requiere Python ≥ 3.10 . No tiene dependencias externas para ejecutarse; solo `pytest` para los tests.

Desde `C:\pythontest`:

```
python -m calculator
```

Sin argumentos ejecuta una batería de ejemplos.

Con argumentos evalúa cada expresión:

```
python -m calculator "(4,2, /)" "[[(1,2), (3,4)], D]"
```

Nota: en PowerShell/Bash conviene entrecomillar cada expresión.

2. Formato general de entrada

La calculadora reconoce **cuatro formatos** según el carácter inicial:

Empieza por	Forma	Uso
(	(arg, arg, ..., OP)	operaciones escalares y polinomios (con X/Y, P)
[	[operando, operando, OP]	polinomios (+,-,*), matrices, escalar×matriz
[	[operando, OP]	operadores unarios sobre matriz: T, D
{	{n1, n2, ..., nk}	suma directa de todos los números

En todos los casos los espacios se ignoran y el **último token** es el operador.

Números aceptados

Token	Valor
123, -4.5, 1e3	Literal float
E o e	math.e (2.71828...)
PI o pi	math.pi (3.14159...)

3. Operaciones binarias escalares (a, b, OP)

OP	Nombre	Ejemplo	Resultado
+	Suma	(1, 1, +)	2
-	Resta	(7, 5, -)	2
*	Producto	(6, 3, *)	18
/	División	(4, 2, /)	2
^	Potencia	(3, 2, ^)	9
R	Raíz enésima (b = índice)	(9, 2, R)	3
L	Logaritmo (b = base)	(100, 10, L)	2

Casos especiales:

- (2, E, *) $\rightarrow 2 \cdot e$
- (PI, 3, *) $\rightarrow 3\pi$
- (10, E, L) $\rightarrow$ logaritmo neperiano de 10
- (-8, 3, R) $\rightarrow -2$ (raíz cúbica de -8)

Errores

- (1, 0, /) $\rightarrow$ ZeroDivisionError: División por cero.
- (-9, 2, R) $\rightarrow$ raíz de índice par de negativo.
- (1, 0, R) $\rightarrow$ índice 0.
- (0, 10, L) o base ≤ 0 o base = 1 $\rightarrow$ logaritmo inválido.

4. Operaciones unarias escalares (a, OP)

OP	Nombre	Ejemplo	Resultado
!	Factorial	(5, !)	120
F	Fibonacci n-ésimo (F ₀ =0, F ₁ =1)	(10, F)	55

Requieren entero ≥ 0 .

5. Ecuaciones polinómicas (a_n, ..., a₀, X)

El operador X (o Y, equivalente) resuelve el polinomio con coeficientes de **mayor a menor grado**.

Entrada	Ecuación	Salida
(1, -3, 2, X)	$x^2 - 3x + 2 = 0$	X1 = 2, X2 = 1
(2, 2, 2, X)	$2x^2 + 2x + 2 = 0$	X1 = -0.5 + 0.866i, X2 = -0.5 - 0.866i
(1, -6, 11, -6, X)	$x^3 - 6x^2 + 11x - 6 = 0$	X1 = 1, X2 = 3, X3 = 2

- Grado 1: despeje directo.
- Grado 2: fórmula cuadrática (raíces complejas si el discriminante es negativo).
- Grado ≥ 3 : **Durand-Kerner** (raíces reales y complejas).
- El coeficiente principal no puede ser 0.

6. Números primos (..., P)

Entrada	Significado	Resultado
(10, P)	Primos ≤ 10	2, 3, 5, 7
(1, 100, P)	Primos en el intervalo [1, 100]	2, 3, 5, 7, ..., 97
(50, 70, P)	Primos en [50, 70]	53, 59, 61, 67

- Implementado con criba de Eratóstenes (bytearray).
- **Límite superior:** 100 000 000 (100 M).

7. Aritmética de polinomios [(p), (q), OP]

Entrada	Cálculo	Resultado
[(1, 1), (1, -1), *]	$(x+1)(x-1)$	(1, 0, -1)
[(1, 1), (1, 1), *]	$(x+1)^2$	(1, 2, 1)
[(1, 2, 3), (4, 5, 6), +]	suma coef. a coef.	(5, 7, 9)
[(1, 2, 3), (1, 1), -]	$(x^2+2x+3)-(x+1)$	(1, 1, 2)

- Suma/resta alinean por el término independiente.
- Ceros a la izquierda del resultado se eliminan.

8. Aritmética de matrices [[m1], [m2], OP]

Una matriz se escribe con **corchetes exteriores** y una tupla (...) por fila.

$[(1, 1), (1, 1)], [(1, 1), (1, 1)], +] \rightarrow [(2, 2), (2, 2)]$
 $[(1, 2), (3, 4)], [(5, 6), (7, 8)], -] \rightarrow [(-4, -4), (-4, -4)]$
 $[(1, 2), (3, 4)], [(5, 6), (7, 8)], *] \rightarrow [(19, 22), (43, 50)]$

Vectores

- **Vector fila:** [(a, b, c)] (matriz 1×n).
- **Vector columna:** [(a), (b), (c)] (matriz n×1).

Reglas

- +/−: dimensiones idénticas.

- *: cols(A) == rows(B). Resultado: rows(A) × cols(B).

9. Escalar × matriz [k, [m], *] o [[m], k, *]

$[3, [(1, 2), (3, 4)], *] \rightarrow [(3, 6), (9, 12)]$
 $[[[(1, 2), (3, 4)], 2], *] \rightarrow [(2, 4), (6, 8)]$

10. Traspuesta [[m], T]

$[[[(1, 2, 3), (4, 5, 6)], T] \rightarrow [(1, 4), (2, 5), (3, 6)]$

11. Determinante [[m], D]

$[[[(1, 2), (3, 4)], D] \rightarrow -2$
 $[[[(6, 1, 1), (4, -2, 5), (2, 8, 7)], D] \rightarrow -306$

Eliminación gaussiana con pivoteo parcial, $O(n^3)$. Matriz no cuadrada → ValueError.

12. Resumen de operadores

Escalares (...)

OP	Aridad	Descripción
+	binario	suma
-	binario	resta
*	binario	producto
/	binario	división
^	binario	potencia
R	binario	raíz de índice b
L	binario	logaritmo base b
!	unario	factorial
F	unario	n-ésimo Fibonacci
P	1 o 2 args	primos ≤ n o en [a, b]
X	≥ 2 args	raíces del polinomio
Y	≥ 2 args	alias de X

Estructuras [...]

OP	Aridad	Operandos	Descripción
+	binario	2 polinomios / 2 matrices	suma

OP	Aridad	Operandos	Descripción
-	binario	2 polinomios / 2 matrices	resta
*	binario	2 polinomios / 2 matrices	producto (convolución / matricial)
*	binario	escalar + matriz	producto escalar por matriz
T	unario	1 matriz	traspuesta
D	unario	1 matriz cuadrada	determinante

13. API pública

```
from calculator import evaluate, solve, process
```

```
evaluate("(4,2,/)")          # 2.0
solve("(1,-3,2,X)")          # [(2+0j), (1+0j)]
process("[[(1,2),(3,4)],D]")  # "-2"
```

- `evaluate(expr)` → `float`. Solo para operaciones escalares (binarias y unarias). `RuntimeError` para entradas polinómicas.
- `solve(expr)` → `list[complex]`. Solo para ecuaciones X/Y.
- `process(expr)` → `str`. **Dispatcher universal**: elige el modo adecuado y formatea.

14. Manejo de errores

Todas las excepciones son subclases de `Exception`:

- `ValueError` — formato inválido, operador desconocido, número no parseable, dimensiones incompatibles, matriz mal formada.
- `ArithmeticError` — raíz par de negativo, logaritmo inválido, factorial/Fibonacci de no entero o negativo, primos con límite $> 10^8$.
- `ZeroDivisionError` — división por cero.
- `RuntimeError` — llamar a `evaluate()` con una entrada polinómica.

El `__main__` incorpora `try/except` que imprime `ERROR: <mensaje>`.

15. Tests

```
python -m pytest tests/ -q
```

42 tests que cubren operadores binarios/unarios, primos, ecuaciones, polinomios, matrices y formatos malformados.

16. Suma de array {n1, n2, ..., nk}

Formato para sumar una lista arbitraria de números. Disponible tanto en la versión completa como en la reducida.

Entrada	Resultado
{1, 2, 3, 4, 5}	15
{PI, E}	$\pi + e$
{10, -3, 2.5}	9.5
{}	0

Acepta literales numéricos y las constantes E/PI. Errores: falta } de cierre → ValueError.

17. Versión reducida

calculator_free.py es un script autónomo que ofrece únicamente +, -, *, / sobre operandos (a, b, op) (con E/PI como constantes) y la suma de array {n1, ..., nk}:

```
python calculator_free.py "(2,E,*)"
```